

I Service in Android

e come PosMock li usa

PosMock — simulatore di terminale POS · 14/08/2026

Questo documento risponde a due domande messe in fila. La prima e' generale: **che cos'e' davvero un Service in Android**, cosa risolve e perche' la piattaforma ha continuato a stringerne le regole a ogni versione. La seconda e' specifica: **perche' PosMock ne ha uno**, come e' costruito e — soprattutto — perche' il server TCP che simula il terminale *non* vive dentro di lui.

La seconda parte e' la piu' utile da rileggere fra sei mesi: le scelte fatte qui sembrano arbitrarie finche' non si ricorda il guasto che dovevano evitare.

1. Che cos'e' un Service

Un **Service** e' un componente dell'applicazione, dichiarato nel manifest, che svolge un lavoro **senza interfaccia utente** e con una vita indipendente da quella delle Activity. Un'Activity muore quando l'utente la lascia; un Service no.

Le tre cose che un Service non e'

Quasi tutti gli errori con i Service nascono da uno di questi tre malintesi.

- **Non e' un thread.** Un Service gira sul *main thread* del processo, esattamente come una Activity. Metterci dentro una lettura da socket bloccante congela la UI. Il lavoro in background va comunque messo su una coroutine o un thread proprio.
- **Non e' un processo separato.** Vive nello stesso processo dell'app, quindi condivide memoria, singleton e stato statico. (Si puo' forzare un processo diverso con `android:process`, ma allora smette di condividere tutto ed e' un'altra storia.)
- **Non e' "codice che gira per sempre".** Il sistema puo' ucciderlo. Quello che un Service compra e' *priorita'*, non immortalita'.

Detta nel modo che conta: un Service e' il modo di **dichiarare al sistema che il processo sta facendo qualcosa che vale anche senza nessuno che guardi lo schermo**. Tutto il resto discende da qui.

2. Il problema che risolve: il low memory killer

Android non ha swap. Quando la memoria scarseggia, il sistema uccide processi, e li sceglie per **rango di importanza**. Il rango non lo decide l'app: lo deduce il sistema da quali componenti sono vivi al suo interno.

Rango del processo	Quando	Rischio di essere ucciso
Foreground	Activity in primo piano, oppure un foreground service	Quasi nullo: solo in emergenza estrema
Visible	Activity visibile ma non a fuoco (dietro un dialog)	Molto basso
Service	Un service <i>background</i> avviato con <code>startService</code>	Medio
Cached / Empty	Nessun componente attivo, tenuto solo per riaprirlo in fretta	Alto: e' il primo della lista

Un'app senza componenti attivi che finisce in background scivola subito in *cached*. Da lì viene uccisa senza preavviso, senza callback, senza nulla: le socket aperte si chiudono, le coroutine spariscono. Il **foreground service** e' l'unico strumento che porta il processo in cima a quella lista mentre l'utente sta guardando altro.

3. Started e Bound: due modi di usarlo

	Started service	Bound service
Come parte	<code>startService()</code> / <code>startForegroundService()</code>	<code>bindService()</code>
Chi lo tiene vivo	Se stesso, finché non chiama <code>stopSelf()</code> o qualcuno <code>stopService()</code>	I client collegati: quando l'ultimo si stacca, muore
Comunicazione	Intent in ingresso; per uscire servono broadcast, callback o stato condiviso	Un <code>IBinder</code> : chiamate dirette ai metodi
Usato per	Lavoro che deve finire a prescindere dalla UI	Una UI che comanda un componente e ne legge lo stato

PosMock usa il primo e ignora il secondo: `onBind()` ritorna `null`. Il perché e' nel capitolo 7 — la UI non ha nulla da chiedere al service.

4. Perché oggi “foreground” non e' una scelta stilistica

Fino ad Android 7 un service in background poteva vivere indefinitamente. E' stato abusato al punto che ogni versione successiva ha aggiunto un vincolo. Vale la pena conoscerli, perché spiegano quasi tutte le righe del nostro manifest.

Versione	Vincolo introdotto
8.0 (API 26)	Background execution limits : un service avviato mentre l'app e' in background viene fermato dopo circa un minuto. Nasce <code>startForegroundService()</code> , che obbliga a chiamare <code>startForeground()</code> con una notifica entro 5 secondi , pena una ANR.
10 (API 29)	Compare <code>foregroundServiceType</code> per l'accesso a posizione, camera e microfono.
12 (API 31)	Vietato avviare un foreground service mentre l'app e' in background, salvo eccezioni elencate.
13 (API 33)	<code>POST_NOTIFICATIONS</code> diventa un permesso runtime: senza, il service parte ma la sua notifica non si vede.
14 (API 34)	<code>foregroundServiceType</code> obbligatorio per ogni FGS, con un permesso dedicato per tipo. Il tipo va passato anche a <code>startForeground()</code> .
15 (API 35)	Il tipo <code>dataSync</code> ha un tetto di 6 ore complessive ogni 24 , dopo le quali il sistema lo ferma.

Quando invece non serve un Service. Per lavoro *differibile* — sincronizzare, caricare, ripulire — la risposta giusta e' **WorkManager**: sopravvive al riavvio del telefono e rispetta i vincoli di batteria. Il Service serve quando il lavoro e' *continuo, adesso, e non puo' essere rimandato*. Il nostro caso e' esattamente questo: una socket in ascolto non e' rinviabile a quando il telefono sara' in carica.

5. Ciclo di vita e valore di ritorno di onStartCommand

```
onCreate()          una sola volta, alla prima creazione
|
onStartCommand()    a OGNI startService(), anche a service gia' vivo
|                  ritorna una politica di riavvio (tabella qui sotto)
|
|                  ... il service vive; se non fa niente, non "gira": sta solo li'
|
onDestroy()         su stopSelf(), stopService(), o kill del sistema
```

Il valore ritornato da `onStartCommand()` dice al sistema cosa fare se il processo viene ucciso lo stesso.

Valore	Se il sistema uccide il processo
START_STICKY	Ricrea il service appena puo', con un intent nullo . Adatto a chi sa ripartire da solo (un player di musica).
START_NOT_STICKY	Non lo ricrea. Adatto a chi, senza il proprio stato in memoria, non saprebbe che farsene di una seconda vita.
START_REDELIVER_INTENT	Lo ricrea riconsegnando l'ultimo intent . Adatto a un lavoro descritto per intero dall'intent (un download).

6. PosMock: il guasto da cui nasce tutto

PosMock finge di essere un terminale POS. Apre un `ServerSocket` e risponde al middleware come farebbe un terminale vero, cosi' da poter provare il percorso di pagamento senza hardware — e soprattutto da poter **riprodurre a comando** i casi che sul banco non si riescono a provocare.

```
Giano (cassa) <-> Hermes (palmare) --WS--> UniquePosManager --TCP--> PosMock
                                     (invece del POS vero)
```

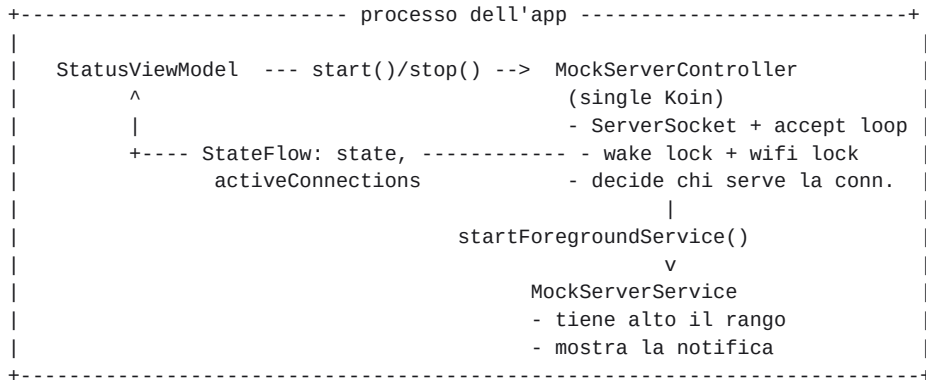
Il banco di prova resta acceso a lungo: un telefono appoggiato alla scrivania, la sessione di test che dura un turno intero, lo schermo che si spegne fra una prova e l'altra. E qui c'e' la trappola.

Se il processo di PosMock muore, la socket si chiude a meta' transazione. Il middleware vede una connessione caduta e un esito che non arriva: **esattamente i sintomi del pagamento orfano**, cioe' del guasto che si sta indagando. Si passerebbero ore a cercare il bug nel middleware mentre il colpevole e' il telefono andato in Doze. Un banco di prova che produce da solo il sintomo che deve misurare non e' uno strumento: e' una fonte di errori.

Il foreground service esiste per questo, e per nient'altro. Non e' una scelta di architettura: e' una difesa contro un falso positivo.

7. La decisione centrale: il server non vive nel Service

L'istinto sarebbe mettere l'accept loop dentro MockServerService. In PosMock e' l'opposto: il server e' **MockServerController**, un single Koin che vive quanto il processo, e il service e' un guscio che gli sta accanto.



Cosa si guadagna

- **La UI legge lo stato senza intermediari.** StatusViewModel osserva direttamente gli StateFlow del controller. Niente IBinder, niente broadcast, niente serializzazione dello stato dentro e fuori da un Intent.
- **Il controllo e' invertito.** Non e' il service ad accendere il server: e' il controller che, appena la socket e' in bind, chiama startForegroundService(), e che in stop() lo spegne. Il ciclo di vita del service segue quello del server, non viceversa.
- **Il server e' testabile e indipendente da Android.** La logica di protocollo non sa nemmeno che esista un service.

```

// MockServerController.start() -- ordine non casuale
serverJob = scope.launch {
    socket = ServerSocket().apply { reuseAddress = true; bind(...) }
    acquireLocks()           // wake lock + wifi lock: PRIMA di dichiararsi pronto
    startService()           // il foreground service, ora che la porta e' davvero aperta
    _state.value = ServerState.Running(config.protocol, config.port)
    while (isActive) { ... accept() ... }
}

private fun startService() {
    appContext.startForegroundService(Intent(appContext, MockServerService::class.java))
}

```

Il service parte **dopo** il bind riuscito: se la porta fosse occupata comparirebbe una notifica "in ascolto" su un server che non c'e'.

8. Il Service, riga per riga

La dichiarazione nel manifest

```

<service
    android:name=".service.MockServerService"
    android:exported="false"
    android:foregroundServiceType="specialUse">
    <property
        android:name="android.app.PROPERTY_SPECIAL_USE_FGS_SUBTYPE"
        android:value="Simulatore di terminale POS per test di integrazione: tiene
        aperto un server TCP mentre lo schermo e' spento" />
</service>

```

- **exported="false"**: nessuna altra app deve poterlo avviare.

- **specialUse** e non `dataSync`: come visto al capitolo 4, su Android 15 un FGS `dataSync` viene fermato dopo 6 ore complessive nelle 24. Un banco di prova puo' restare acceso per un turno intero, quindi quel tetto lo taglierebbe a meta' pomeriggio. `specialUse` non ha il tetto, ma richiede la `<property>` che ne descrive il motivo — e su Google Play una revisione manuale. PosMock e' uno strumento interno e non passa dallo store, quindi il compromesso conviene.

onCreate: notifica entro cinque secondi

```
override fun onCreate() {
    super.onCreate()
    createChannel()
    startForegroundWithNotification(buildNotification("Avvio in corso..."))
    observeState()
}

private fun startForegroundWithNotification(notification: Notification) {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.UPSIDE_DOWN_CAKE) {
        startForeground(NOTIFICATION_ID, notification, FOREGROUND_SERVICE_TYPE_SPECIAL_USE)
    } else {
        startForeground(NOTIFICATION_ID, notification)
    }
}
```

La notifica iniziale dice “Avvio in corso...” perche' la promessa fatta a `startForegroundService()` va mantenuta subito: il testo definitivo arriva un istante dopo dallo `StateFlow`. Da Android 14 il tipo va ripetuto anche qui, non basta il `manifest`.

La notifica si aggiorna da sola

```
private fun observeState() {
    scope.launch {
        serverController.state.collectLatest { state ->
            when (state) {
                is ServerState.Running -> updateNotification(
                    "${state.protocol.label} in ascolto sulla porta ${state.port}"
                )
                ServerState.Starting -> updateNotification("Avvio in corso...")
                ServerState.Stopped -> updateNotification("Server fermo")
                is ServerState.Error -> reportErrorAndStop(state.message)
            }
        }
    }
}
```

Il service **consuma** lo stesso `StateFlow` che alimenta la UI. Non lo produce, non lo duplica: c'e' una sorgente di verita' sola, ed e' il controller. La notifica e' una seconda vista sullo stesso dato.

Il ramo `Error` e' l'unico che non aggiorna e basta. Se il server muore per conto suo — l'interfaccia di rete che cade mentre il banco e' acceso — il service non ha piu' niente da tenere vivo e si ferma; ma prima lascia in tendina una notifica d'errore **con un id diverso**, perche' quella del foreground service viene portata via dal sistema insieme al service. E' scartabile, a differenza dell'altra, perche' non c'e' piu' niente da fermare.

Un banco che sparisce in silenzio e' il problema, non la soluzione. Senza quella seconda notifica, una caduta di rete lascerebbe il telefono muto e apparentemente a posto: si tornerebbe a cercare il guasto nel middleware. Per lo stesso motivo il controller, nel ramo d'errore, rilascia subito `wake lock` e `wifi lock` — tenerli per un server che non ascolta piu' e' solo batteria buttata.

L'azione “Ferma”

```
val stop = PendingIntent.getService(
    this, 1,
    Intent(this, MockServerService::class.java).apply { action = ACTION_STOP },
    PendingIntent.FLAG_IMMUTABLE or PendingIntent.FLAG_UPDATE_CURRENT,
)

override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
    if (intent?.action == ACTION_STOP) {
        scope.launch {
            serverController.stop() // il service non spegne il server: glielo chiede
            stopSelf()
        }
        return START_NOT_STICKY
    }
    return START_NOT_STICKY
}
```

- Il banco si spegne dalla tendina, senza riaprire l'app: comodo quando il telefono e' gia' appoggiato con lo schermo bloccato.
- **FLAG_IMMUTABLE** e' obbligatorio da Android 12 (API 31): un PendingIntent mutabile potrebbe essere riempito da altri.
- L'azione passa comunque da `serverController.stop()`, che chiude le socket, rilascia i lock e aggiorna lo stato. Se il service chiudesse per conto suo, la UI mostrerebbe un server acceso che non esiste piu'.
- `stop()` e' **sospensibile**, perche' aspetta davvero che l'accept loop sia finito: al ritorno la porta e' libera per un riavvio immediato. Da qui il `launch` — e il `stopSelf()` messo *dopo*, dato che l'onDestroy del service cancellerebbe lo scope in cui quella pulizia sta girando.
- Dentro, la pulizia gira in `NonCancellable`: gli scope che la ospitano muoiono per conto loro — il `viewModelScope` alla rotazione dello schermo — e una pulizia interrotta a meta' lascerebbe il wake lock in mano.

Perche' START_NOT_STICKY

E' la riga che sorprende di piu', e ha una ragione precisa. Se malgrado tutto il sistema uccide il processo, **il server muore con lui**: la socket, la configurazione in memoria, le connessioni aperte, la coroutine dell'accept loop. Un service resuscitato da solo si ritroverebbe senza niente da presidiare e mostrerebbe una notifica “in ascolto” davanti a una porta chiusa.

Meglio un banco spento e visibile che una notifica che mente. Con `START_NOT_STICKY` la sparizione e' evidente e l'operatore riavvia; con `START_STICKY` sarebbe silenziosa, e si tornerebbe a cercare il guasto dalla parte sbagliata — cioe' proprio nel modo che questo service doveva impedire.

9. Il Service da solo non basta: le quattro difese

Il foreground service tiene alto il rango del processo, ma non impedisce alla CPU di sospendersi ne' al Wi-Fi di andare in risparmio energetico. Servono tutte e quattro le protezioni insieme, le stesse gia' collaudate in Hermes.

Difesa	Contro cosa	Dove
Foreground service specialUse	Il processo ucciso con l'app in background	MockServerService
Wake lock parziale	La CPU sospesa a schermo spento	MockServerController
Wifi lock FULL_LOW_LATENCY (FULL_HIGH_PERF sotto API 29)	Il Wi-Fi in risparmio energetico: il middleware trova un terminale che non risponde	MockServerController
FLAG_KEEP_SCREEN_ON + esenzione dalle ottimizzazioni di batteria	Doze e gli OEM aggressivi (Samsung One UI in testa), che sospendono lo stesso	MainActivity

Lock presi e rilasciati insieme al server, non insieme al service: stanno in `acquireLocks()` / `releaseLocks()` del controller, per lo stesso motivo per cui li sta la socket.

L'esenzione dalla batteria viene richiesta a **ogni avvio** finché non è concessa, perché la sorgente di verità è lo stato del sistema (`isExempt()`) e non una preferenza nostra: un reset fatto dall'OEM la toglierebbe senza dirlo a nessuno.

10. Le alternative, e perché sono state scartate

Alternativa	Perché no
Nessun service, solo l'Activity	Con lo schermo spento il processo scivola in <i>cached</i> e viene ucciso. È esattamente il guasto da evitare.
WorkManager	Fatto per lavoro differibile e a scadenza. Una socket in ascolto non è rinviabile a quando il telefono sarà in carica.
Bound service con IBinder	Aggiungerebbe binder, callback e gestione della disconnessione per ottenere quello che due StateFlow su un singleton danno gratis.
Il server dentro il service	Il ciclo di vita della socket seguirebbe quello di un componente Android, e la UI dovrebbe parlarci via binder o broadcast. Più pezzi, stesso risultato.
dataSync invece di specialUse	Tetto di 6 ore ogni 24 su Android 15: il banco si spegnerebbe da solo a metà giornata di prove.

11. Riepilogo: chi fa che cosa

File	Responsabilità
<code>data/server/MockServerController.kt</code>	Il server vero. ServerSocket, accept loop, wake e wifi lock, scelta del protocol handler, avvio e arresto del service. Singleton di processo.
<code>service/MockServerService.kt</code>	Solo due cose: tiene alto il rango del processo e mostra la notifica con l'azione Ferma. Nessuna logica di rete.
<code>presentation/.../StatusViewModel.kt</code>	Chiama <code>start()</code> / <code>stop()</code> sul controller e ne osserva gli StateFlow. Non conosce il service.
<code>presentation/activity/MainActivity.kt</code>	FLAG_KEEP_SCREEN_ON, permesso notifiche, richiesta di esenzione dalla batteria.
<code>AndroidManifest.xml</code>	Dichiarazione del service, tipo specialUse con la sua property, permessi di foreground service e wake lock.
<code>di/AppModule.kt</code>	Tutti single: un factory significherebbe due server a contendersi la stessa porta.

In una riga: il Service non serve a far girare il server — serve a impedire che Android lo faccia sparire nel momento peggiore, e a lasciare in tendina un interruttore per spegnerlo.